

Тема 2.5 Функции

Функция – это фрагмент кода, который можно вызвать из любого места программы. Например, с помощью функции **len()** можно получить количество элементов последовательности. Начиная с этого шага, рассмотрим создание пользовательских функций, которые позволят уменьшить избыточность программного кода и повысить его структурированность.

Определение и вызов функции

Функция создается (или, как говорят программисты, определяется) с помощью ключевого слова **def** по следующей схеме:

```
def <Имя функции> ( [ <Параметры> ] ) :  
    [ """ Строка документирования """ ]  
    <Тело функции>  
    [return <Результат>]
```

Имя функции должно быть уникальным идентификатором, состоящим из латинских букв, цифр и знаков подчеркивания, причем имя функции не может начинаться с цифры. В качестве имени нельзя использовать ключевые слова, кроме того, следует избегать совпадений с названиями встроенных идентификаторов. Регистр символов в названии функции также имеет значение.

После имени функции в круглых скобках можно указать один или несколько параметров через запятую, а если функция не принимает параметры, указываются только круглые скобки. После круглых скобок ставится двоеточие.

Тело функции представляет собой составную конструкцию. Как и в любой составной конструкции, инструкции внутри функции выделяются одинаковым количеством пробелов слева. Концом функции считается инструкция, перед которой находится меньшее количество пробелов. Если тело функции не содержит инструкций, то внутри ее необходимо разместить оператор **pass**, который не выполняет никаких действий. Этот оператор удобно использовать на этапе отладки программы, когда определили функцию, а тело решили дописать позже.

Пример функции, которая ничего не делает:

```
def func(): pass
```

Необязательная инструкция **return** позволяет вернуть из функции какое-либо значение в качестве результата. После исполнения этой инструкции выполнение функции будет остановлено. Это означает, что инструкции, следующие после оператора **return**, никогда не будут выполнены.

Пример:

```
def func():  
    print("Текст до инструкции return")  
    return "Возвращаемое значение"  
    print("Эта инструкция никогда не будет выполнена")  
print(func()) # Вызываем функцию
```

Результат выполнения:

Текст до инструкции return

Возвращаемое значение

Инструкции **return** может не быть вообще. В этом случае выполняются все инструкции внутри функции, и в качестве результата возвращается значение **None**.

Для примера создадим три функции:

```
def print_ok():
    """ Пример функции без параметров """
    print("Сообщение при удачно выполненной операции")
def echo(m):
    """ Пример функции с параметром """
    print(m)
def summa(x, y):
    """ Пример функции с параметрами,
        возвращающей сумму двух переменных """
    return x + y
```

При вызове функции значения передаются внутри круглых скобок через запятую. Если функция не принимает параметров, то указываются только круглые скобки. Необходимо также заметить, что количество параметров в определении функции должно совпадать с количеством параметров при вызове, иначе будет выведено сообщение об ошибке. Вызвать созданные функции можно способами, указанными ниже:

```
print_ok()          # Вызываем функцию без параметров
echo("Сообщение")   # Функция выведет сообщение
x = summa(5, 2)      # Переменной x будет присвоено значение 7
a, b = 10, 50
y = summa(a, b)      # Переменной y будет присвоено значение 60
```

Как видно из последнего примера, имя переменной в вызове функции может не совпадать с именем переменной в определении функции. Кроме того, глобальные переменные **x** и **y** не конфликтуют с одноименными переменными в определении функции, т. к. они расположены в разных областях видимости. Переменные, указанные в определении функции, являются локальными и доступны только внутри функции.

Оператор **+**, используемый в функции **summa()**, применяется не только для сложения чисел, но и позволяет объединить последовательности. То есть, функция **summa()** может использоваться не только для сложения чисел. В качестве примера выполним конкатенацию строк и объединение списков:

```
def summa(x, y):
    return x + y
print(summa("str", "ing")) # Выведет: string
print(summa([1, 2], [3, 4])) # Выведет: [1, 2, 3, 4]
```

Все в языке Python представляет собой объекты: строки, списки и даже сами типы данных. Не являются исключением и функции. Инструкция **def** создает объект, имеющий тип **function**, и сохраняет ссылку на него в идентификаторе, указанном после инструкции **def**. Таким образом, можно сохранить ссылку на функцию в другой переменной - для этого название функции указывается без круглых скобок. Сохраним ссылку в переменной и вызовем функцию через нее:

```
def summa(x, y):
```

```

    return x + y
f = summa # Сохраняем ссылку в переменной f
v = f(10, 20) # Вызываем функцию через переменную f

```

Можно также передать ссылку на функцию в качестве параметра другой функции. Функции, передаваемые по ссылке, обычно называются **функциями обратного вызова**:

```

def summa(x, y):
    return x + y
def func(f, a, b):
    """ Через переменную f будет доступна ссылка на
        функцию summa() """
    return f(a, b) # Вызываем функцию summa ()
# Передаем ссылку на функцию в качестве параметра
v = func(summa, 10, 20)

```

Объекты функций поддерживают множество атрибутов, обратиться к которым можно, указав атрибут после названия функции через точку. Например, через атрибут **__name__** можно получить название функции в виде строки, через атрибут **__doc__** - строку документирования и т. д. Для примера выведем названия всех атрибутов функции с помощью встроенной функции **dir()**:

```

def summa(x, y) :
    """ Суммирование двух чисел """
    return x + y

dir (summa)
['_annotations_', '__call__', '__class__', '__closure__', '__code__',
 '__defaults__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__get__', '__getattr__', '__globals__',
 '__gt__', '__hash__', '__init__', '__kwdefaults__', '__le__', '__lt__',
 '__module__', '__name__', '__ne__', '__new__', '__qualname__', '__reduce__',
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__',
 '__subclasshook__']
print(summa.__code__.co_varnames)
('x', 'y')
print(summa.__doc__)
'Суммирование двух чисел '

```

Расположение определений функций

Все инструкции в программе выполняются последовательно сверху вниз. Это означает, что, прежде чем использовать в программе идентификатор, его необходимо предварительно определить, присвоив ему значение. Поэтому определение функции должно быть расположено перед вызовом функции.

Правильно:

```

def summa(x, y):
    return x + y
v = summa(10, 20) # Вызываем после определения. Все нормально

```

Неправильно:

```
v = somma(10, 20) # Идентификатор еще не определен. Это ошибка!!!
def somma(x, y):
    return x + y
```

В последнем случае будет выведено сообщение об ошибке: **NameError: name 'somma' is not defined**. Чтобы избежать ошибки, определение функции размещают в самом начале программы после подключения модулей или в отдельном модуле.

С помощью оператора ветвления **if** можно изменить порядок выполнения программы - например, разместить внутри условия несколько определений функций с одинаковым названием, но разной реализацией:

```
# -*- coding: utf-8 -*-
n = input("Введите 1 для вызова первой функции: ")
if n == "1":
    def echo():
        print("Вы ввели число 1")
else:
    def echo():
        print("Альтернативная функция")
echo() # Вызываем функцию
input()
```

При вводе числа 1 мы получим сообщение *"Вы ввели число 1"*, в противном случае - *"Альтернативная функция"*.

Помните, что инструкция **def** всего лишь присваивает ссылку на объект функции идентификатору, расположенному после ключевого слова **def**. Если определение одной функции встречается в программе несколько раз, то будет использоваться функция, которая была определена последней. Пример:

```
def echo():
    print("Вы ввели число 1")
def echo():
    print("Альтернативная функция")
echo() # Всегда выводит "Альтернативная функция"
```

Необязательные параметры и сопоставление по ключам

Чтобы сделать некоторые параметры необязательными, следует в определении функции присвоить этому параметру начальное значение. Переделаем функцию суммирования двух чисел и сделаем второй параметр необязательным.

```
def somma(x, y=2): # y - необязательный параметр
    return x + y
a = somma(5) # Переменной a будет присвоено значение 7
b = somma(10, 50) # Переменной b будет присвоено значение 60
```

Таким образом, если второй параметр не задан, то его значение будет равно 2. Обратите внимание на то, что необязательные параметры должны следовать после обязательных параметров, иначе будет выведено сообщение об ошибке.

До сих пор мы использовали позиционную передачу параметров в функцию:

```
def summa(x, y):
    return x + y
print(summa(10, 20)) # Выведет: 30
```

Переменной `x` при сопоставлении будет присвоено значение 10, а переменной `y` - значение 20. Но язык **Python** позволяет также передать значения в функцию, используя сопоставление по ключам. Для этого при вызове функции параметрам присваиваются значения, причем последовательность указания параметров в этом случае может быть произвольной:

```
def summa(x, y):
    return x + y
print(summa(y=20, x=10)) # Сопоставление по ключам
```

Сопоставление по ключам очень удобно использовать, если функция имеет несколько необязательных параметров. В этом случае не нужно перечислять все значения, а достаточно присвоить значение нужному параметру. Пример:

```
def summa(a=2, b=3, c=4): # Все параметры являются необязательными
    return a + b + c
print(summa(2, 3, 20)) # Позиционное присваивание
print(summa(c=20)) # Сопоставление по ключам
```

Если значения параметров, которые планируется передать в функцию, содержатся в кортеже или списке, то перед объектом следует указать символ `*`. Пример передачи значений из кортежа и списка приведен ниже:

```
def summa(a, b, c):
    return a + b + c
t1, arr = (1, 2, 3), [1, 2, 3]
print(summa(*t1)) # Распаковываем кортеж
print(summa(*arr)) # Распаковываем список
t2 = (2, 3)
print(summa(1, *t2)) # Можно комбинировать значения
```

Если значения параметров содержатся в словаре, то распаковать словарь можно, указав перед ним две звездочки (`**`):

```
def summa(a, b, c):
    return a + b + c
d1 = {"a": 1, "b": 2, "c": 3}
print(summa(**d1)) # Распаковываем словарь
t, d2 = (1, 2), {"c": 3}
print(summa(*t, **d2)) # Можно комбинировать значения
```

Объекты в функцию передаются по ссылке. Если объект относится к неизменяемому типу, то изменение значения внутри функции не затронет значение переменной вне функции:

```
def func(a, b):
```

```

a, b = 20, "str"
x, s = 80, "test"
func(x, s) # Значения переменных x и s не изменяются
print(x, s) # Выведет: 80 test

```

В этом примере значения в переменных `x` и `s` не изменились. Однако если объект относится к изменяемому типу, то ситуация будет другой:

```

def func(a, b):
    a[0], b["a"] = "str", 800

```

```

x = [1, 2, 3] # Список
y = {"a": 1, "b": 2} # Словарь
func(x, y) # Значения будут изменены!!!
print(x, y) # Выведет: ['str', 2, 3] {'a': 800, 'b': 2}

```

Как видно из примера, значения в переменных `x` и `y` изменились, поскольку список и словарь относятся к изменяемым типам. Чтобы избежать изменения значений, внутри функции следует создать копию объекта:

```

def func(a, b):
    a = a[:] # Создаем поверхностную копию списка
    b = b.copy() # Создаем поверхностную копию словаря
    a[0], b["a"] = "str", 800
x = [1, 2, 3] # Список
y = {"a": 1, "b": 2} # Словарь
func(x, y) # Значения останутся прежними
print(x, y) # Выведет: [1, 2, 3] {'a': 1, 'b': 2}

```

Можно также сразу передавать копию объекта в вызове функции:

```

func(x[:], y.copy())

```

Если указать объект, имеющий изменяемый тип, в качестве значения по умолчанию, то этот объект будет сохраняться между вызовами функции. Пример:

```

def func(a=[]):
    a.append(2)
    return a
print(func()) # Выведет: [2]
print(func()) # Выведет: [2, 2]
print(func()) # Выведет: [2, 2, 2]

```

Как видно из примера, значения накапливаются внутри списка. Обойти эту проблему можно, например, следующим образом:

```

def func(a=None): # Создаем новый список, если значение равно None
    if a is None:
        a = []
    a.append(2)
    return a
print(func()) # Выведет: [2]
print(func([1])) # Выведет: [1, 2]
print(func()) # Выведет: [2]

```

Переменное число параметров в функции

Если перед параметром в определении функции указать символ *, то функции можно будет передать произвольное количество параметров. Все переданные параметры сохраняются в кортеже. Для примера напомним функцию суммирования произвольного количества чисел:

```
def summa(*t):
    """ Функция принимает произвольное количество параметров """
    res = 0
    for i in t: # Перебираем кортеж с переданными параметрами
        res += i
    return res
print(summa(10, 20)) # Выведет: 30
print(summa(10, 20, 30, 40, 50, 60)) # Выведет: 210
```

Можно также вначале указать несколько обязательных параметров и параметров, имеющих значения по умолчанию:

```
def summa(x, y=5, *t): # Комбинация параметров
    res = x + y
    for i in t: # Перебираем кортеж с переданными параметрами
        res += i
    return res
print(summa(10)) # Выведет: 15
print(summa(10, 20, 30, 40, 50, 60)) # Выведет: 210
```

Если перед параметром в определении функции указать две звездочки (**), то все именованные параметры будут сохранены в словаре:

```
def func(**d):
    for i in d: # Перебираем словарь с переданными параметрами
        print("{0} => {1}".format(i, d[i]), end=" ")
func(a=1, b=2, c=3) # Выведет: a => 1 c => 3 b => 2
```

При комбинировании параметров параметр с двумя звездочками указывается самым последним. Если в определении функции указывается комбинация параметров с одной звездочкой и двумя звездочками, то функция примет любые переданные ей параметры:

```
def func(*t, **d):
    """ Функция примет любые параметры """
    for i in t:
        print(i, end=" ")
    for i in d: # Перебираем словарь с переданными параметрами
        print("{0} => {1}".format(i, d[i]), end=" ")
func(35, 10, a=1, b=2, c=3) # Выведет: 35 10 a => 1 c => 3 b => 2
func(10) # Выведет: 10
func(a=1, b=2) # Выведет: a => 1 b => 2
```

В определении функции можно указать, что некоторые параметры передаются только по именам. Для этого параметры должны указываться после

параметра с одной звездочкой, но перед параметром с двумя звездочками. Именованные параметры могут иметь значения по умолчанию. Пример:

```
def func(*t, a, b=10, **d):
    print(t, a, b, d)
func(35, 10, a=1, c=3) # Выведет: (35, 10) 1 10 {'c': 3}
func(10, a=5) # Выведет: (10,) 5 10 {}
func(a=1, b=2) # Выведет: () 1 2 {}
func(1, 2, 3) # Ошибка. Параметр a обязателен!
```

В этом примере переменная **t** примет любое количество значений, которые будут объединены в кортеж. Переменные **a** и **b** должны передаваться только по именам, причем переменной **a** обязательно нужно передать значение при вызове функции. Переменная **b** имеет значение по умолчанию, поэтому при вызове допускается не передавать ей значение, но если значение передается, то оно должно быть указано после названия параметра и оператора **=**. Переменная **d** примет любое количество именованных параметров и сохранит их в словаре. Обратите внимание на то, что, хотя переменные **a** и **b** являются именованными, они не попадут в этот словарь.

Параметра с двумя звездочками может не быть в определении функции, а вот параметр с одной звездочкой при указании параметров, передаваемых только по именам, должен присутствовать обязательно. Если функция не должна принимать переменного количества параметров, но необходимо использовать переменные, передаваемые только по именам, то можно указать только звездочку без переменной:

```
def func(x=1, y=2, *, a, b=10):
    print(x, y, a, b)
func(35, 10, a=1) # Выведет: 35 10 1 10
func(10, a=5) # Выведет: 10 2 5 10
func(a=1, b=2) # Выведет: 1 2 1 2
func(a=1, y=8, x=7) # Выведет: 7 8 1 10
func(1, 2, 3) # Ошибка. Параметр a обязателен!
```

В этом примере значения переменным **x** и **y** можно передавать как по позициям, так и по именам. Поскольку переменные имеют значения по умолчанию, допускается вообще не передавать им значений. Переменные **a** и **b** расположены после параметра с одной звездочкой, поэтому передать значения при вызове можно только по именам. Так как переменная **b** имеет значение по умолчанию, допускается не передавать ей значение при вызове, а вот переменная **a** обязательно должна получить значение, причем только по имени.

Глобальные и локальные переменные

Глобальные переменные – это переменные, объявленные в программе вне функции. В Python глобальные переменные видны в любой части модуля, включая функции.

```
def func(glob2):
```



```

print("Значение глобальной переменной glob1 =", glob1)
glob2 += 10
print("Значение локальной переменной glob2 =", glob2)
glob1, glob2 = 10, 5
func(77) # Вызываем функцию
print("Значение глобальной переменной glob2 =", glob2)

```

Переменной **glob2** внутри функции присваивается значение, переданное при вызове функции. В результате создается новая переменная с тем же именем: **glob2**, которая является локальной. Все изменения этой переменной внутри функции не затронут значение одноименной глобальной переменной.

Итак, **локальные переменные** — это переменные, объявляемые внутри функций. Если имя локальной переменной совпадает с именем глобальной переменной, то все операции внутри функции осуществляются с локальной переменной, а значение глобальной переменной не изменяется. Локальные переменные видны только внутри тела функции.

```

def func():
    local1 = 77 # Локальная переменная
    glob1 = 25 # Локальная переменная
    print("Значение glob1 внутри функции =", glob1)
glob1 = 10 # Глобальная переменная
func() # Вызываем функцию
print("Значение glob1 вне функции =", glob1)
try:
    print(local1) # Вызовет исключение NameError
except NameError: # Обрабатываем исключение
    print("Переменная local1 не видна вне функции")

```

Как видно из примера, переменная **local1**, объявленная внутри функции **func()**, недоступна вне функции. Объявление внутри функции локальной переменной **glob1** не изменило значения одноименной глобальной переменной.

Если обращение к переменной производится до присваивания значения (даже если существует одноименная глобальная переменная), то будет возбуждено исключение **UnboundLocalError**.

```

def func():
    # Локальная переменная еще не определена
    print(glob1) # Эта строка вызовет ошибку!!!
    glob1 = 25 # Локальная переменная
glob1 = 10 # Глобальная переменная
func() # Вызываем функцию

```

Для того чтобы значение глобальной переменной можно было изменить внутри функции, необходимо объявить переменную глобальной с помощью ключевого слова **global**. Продемонстрируем это на примере.

```

def func():
    # Объявляем переменную glob1 глобальной
    global glob1
    glob1 = 25 # Изменяем значение глобальной переменной

```

```

print("Значение glob1 внутри функции =", glob1)
glob1 = 10 # Глобальная переменная
print("Значение glob1 вне функции =", glob1)
func() # Вызываем функцию
print("Значение glob1 после функции =", glob1)

```

Таким образом, поиск идентификатора, используемого внутри функции, будет производиться в следующем порядке:

- поиск объявления идентификатора внутри функции (в локальной области видимости);
- поиск объявления идентификатора в глобальной области;
- поиск во встроенной области видимости (встроенные функции, классы и т. д.).

При использовании анонимных функций следует учитывать, что при указании внутри функции глобальной переменной будет сохранена ссылка на эту переменную, а не ее значение в момент определения функции:

```

x = 5
# Сохраняется ссылка, а не значение переменной x!
func = lambda: x
x = 80 # Изменили значение
print(func()) # Выведет: 80, а не 5

```

Если необходимо сохранить именно текущее значение переменной, то можно воспользоваться способом, приведенным ниже.

```

x = 5
# Сохраняется значение переменной x
func = (lambda y: lambda: y)(x)
x = 80 # Изменили значение
print(func()) # Выведет: 5

```

В третьей строке примера была определена анонимная функция с одним параметром, возвращающая ссылку на вложенную анонимную функцию. Далее вызывается первая функция с помощью круглых скобок и передаем ей значение переменной `x`. В результате сохраняется текущее значение переменной, а не ссылка на нее.

Сохранить текущее значение переменной можно также, указав глобальную переменную в качестве значения параметра по умолчанию в определении функции.

```

x = 5
# Сохраняется значение переменной x
func = lambda x=x: x
x = 80 # Изменили значение
print(func()) # Выведет: 5

```

Получить все идентификаторы и их значения позволяют следующие функции:

- **globals ()** - возвращает словарь с глобальными идентификаторами;
- **locals ()** - возвращает словарь с локальными идентификаторами.

Пример:

```
def func():
    local1 = 54
    glob2 = 25
    print("Глобальные идентификаторы внутри функции")
    print(sorted(globals().keys()))
    print("Локальные идентификаторы внутри функции")
    print(sorted(locals().keys()))
glob1, glob2 = 10, 88
func()
print("Глобальные идентификаторы вне функции")
print(sorted(globals().keys()))
```

`vars ([<Объект>])` - если вызывается без параметра внутри функции, то возвращает словарь с локальными идентификаторами. Если вызывается без параметра вне функции, то возвращает словарь с глобальными идентификаторами. При указании объекта в качестве параметра возвращает идентификаторы этого объекта (эквивалентно вызову `<Объект>.__dict__`). Пример:

```
def func():
    local1 = 54
    glob2 = 25
    print("Локальные идентификаторы внутри функции")
    print(sorted(vars().keys()))
glob1, glob2 = 10, 88
func()
print("Глобальные идентификаторы вне функции")
print(sorted(vars().keys()))
print("Указание объекта в качестве параметра")
print(sorted(vars(dict).keys()))
print("Альтернативный вызов")
print(sorted(dict.__dict__.keys()))
```

Пример решения задачи на языке программирования Python

Задача. Вводится последовательность вещественных чисел. Известно, что последний элемент последовательности равен 5. Разработайте функцию, подсчитывающую количество положительных чисел и минимальное из них.

Решение задачи:

```
def func(chislo=0):
    kolpol=0
    min_=32767
    while chislo!=5: #Пока число, введенное в цикле, не равно числу 5,
        выполнять тело цикла
        chislo = float(input("Введите число "))
        if chislo>0: #Проверка на положительность очередного введенного
            числа
            kolpol=kolpol+1
```

```

        if chislo < min_: #Реализация алгоритма поиска минимального
            элемента
                min_ = chislo
        return kolpol, min_
k, m = func()
print("Количество положительных чисел =", k)
print("Минимальное из них =", m)

```

Анонимные функции

Помимо обычных, язык Python позволяет использовать анонимные функции, которые также называются **лямбда-функциями**. Анонимная функция описывается с помощью ключевого слова **lambda** по следующей схеме:

```

lambda [<Параметр1>[, ..., <ПараметрN>]]: <Возвращаемое значение>

```

После ключевого слова **lambda** можно указать передаваемые параметры. В качестве параметра **<Возвращаемое значение>** указывается выражение, результат выполнения которого будет возвращен функцией. Как видно из схемы, у таких функций нет имени; по этой причине их и называют анонимными.

В качестве значения анонимная функция возвращает ссылку на объект-функцию, которую можно сохранить в переменной или передать в качестве параметра в другую функцию. Вызвать анонимную функцию можно, как и обычную, с помощью круглых скобок, внутри которых расположены передаваемые параметры. Пример использования анонимных функций приведен ниже:

```

f1 = lambda: 10+20 # Функция без параметров
f2 = lambda x,y: x + y # Функция с двумя параметрами
f3 = lambda x,y,z: x + y + z # Функция с тремя параметрами
print(f1()) # Выведет: 30
print(f2(5, 10)) # Выведет: 15
print(f3(5, 10, 30)) # Выведет: 45

```

Как и в обычных функциях, некоторые параметры анонимных функций могут быть необязательными. Для этого параметрам в определении функции присваивается значение по умолчанию:

```

f = lambda x,y=2: x + y
print(f(5)) # Выведет: 7
print(f(5, 6)) # Выведет: 11

```

Чаще всего анонимную функцию не сохраняют в переменной, а сразу передают в качестве параметра в другую функцию. Например, метод списков **sort()** позволяет указать пользовательскую функцию в параметре **key**. Отсортируем список без учета регистра символов, указав в качестве параметра функцию:

```

arr = ["единица1", "Единый", "Единица2"]
arr.sort (key=lambda s: s.lower())
for i in arr:
    print(i, end=" ")
# Результат выполнения: единица1 Единица2 Единый

```

Функции-генераторы

Функцией-генератором называется функция, которая может возвращать одно значение из нескольких значений на каждой итерации. Приостановить выполнение функции и превратить функцию в генератор позволяет ключевое слово **yield**. В качестве примера напомним функцию, которая возводит элементы последовательности в указанную степень:

```
def func(x, y):
    for i in range(1, x + 1):
        yield i ** y
for n in func(10, 2): print(n, end=" ")
# Выведет: 1 4 9 16 25 36 49 64 81 100
print() # Вставляем пустую строку
for n in func(10, 3): print(n, end=" ")
# Выведет: 1 8 27 64 125 216 343 512 729 1000
```

Функции-генераторы поддерживают метод **__next__()**, который позволяет получить следующее значение. Когда значения заканчиваются, метод возбуждает исключение **StopIteration**. Вызов метода **__next__()** в цикле **for** производится незаметно для нас.

В качестве примера перепишем предыдущую программу и используем метод **__next__()** вместо цикла **for**:

```
def func(x, y):
    for i in range(1, x + 1):
        yield i ** y
i = func(3, 3)
print(i.__next__()) # Выведет: 1 (1 ** 3)
print(i.__next__()) # Выведет: 8 (2 ** 3)
print(i.__next__()) # Выведет: 27 (3 ** 3)
print(i.__next__()) # Исключение StopIteration
```

Получается, что с помощью обычных функций мы можем вернуть все значения сразу в виде списка, а с помощью функций-генераторов - только одно значение за раз. Эта особенность очень полезна при обработке большого количества значений, т. к. не понадобится загружать весь список со значениями в память.

В **Python 3.3** появилась возможность вызвать одну функцию-генератор из другой. Для этого применяется расширенный синтаксис ключевого слова **yield**:

```
yield from <Вызываемая функция-генератор>
```

Рассмотрим следующий пример. Пусть у нас есть список чисел, и нам требуется получить другой список, включающий числа в диапазоне от 1 до каждого из чисел в первом списке. Чтобы создать такой список, мы напомним код, показанный ниже:

```
def gen(l):
    for e in l:
        yield from range(1, e + 1)
```

```
l = [5, 10]
for i in gen([5, 10]): print(i, end = " ")
```

Здесь мы в функции-генераторе **gen** перебираем переданный ей в качестве параметра список и для каждого его элемента вызываем другую функцию-генератор. В качестве последней выступает выражение, создающее диапазон от 1 до значения этого элемента, увеличенного на единицу (чтобы это значение вошло в диапазон). В результате на выходе мы получим вполне ожидаемый результат:

```
1 2 3 4 5 1 2 3 4 5 6 7 8 9 10
```

Усложним задачу, включив в результирующий список числа, умноженные на 2. Код, выполняющий эту задачу, показан ниже:

```
def gen2(n):
    for e in range(1, n + 1):
        yield e * 2
def gen(l):
    for e in l:
        yield from gen2(e)
```

```
l = [5, 10]
for i in gen([5, 10]): print(i, end = " ")
```

Здесь мы вызываем из функции-генератора **gen** написанную нами самими функцию-генератор **gen2**. Она создает диапазон, перебирает все входящие в него числа и возвращает их умноженными на 2. Результат работы приведенного в листинге кода таков:

```
2 4 6 8 10 2 4 6 8 10 12 14 16 18 20
```

Декораторы функций

Декораторы позволяют изменить поведение обычных функций - например, выполнить какие-либо действия перед выполнением функции. Рассмотрим это на примере:

```
def deco(f): # Функция-декоратор
    print("Вызвана функция func()")
    return f # Возвращаем ссылку на функцию
@deco
def func(x):
    return "x = {0}".format(x)
print(func(10))
```

В этом примере перед определением функции **func()** указывается название функции **deco()** с предваряющим символом **@**:

```
@deco
```

Таким образом, функция **deco()** становится декоратором функции **func()**. В качестве параметра функция-декоратор принимает ссылку на функцию, поведение которой необходимо изменить, и должна возвращать ссылку на ту же функцию или какую-либо другую. Предыдущий пример эквивалентен следующему коду:

```
def deco(f):
```

```

    print("Вызвана функция func()")
    return f
def func(x):
    return "x = {0}".format(x)
# Вызываем функцию func() через функцию deco()
print (deco(func) (10))

```

Перед определением функции можно указать сразу несколько функций-декораторов. Для примера обернем функцию **func()** в два декоратора: **deco1()** и **deco2()**:

```

def deco1(f):
    print("Вызвана функция deco1()")
    return f
def deco2(f):
    print("Вызвана функция deco2()")
    return f
@deco1
@deco2
def func (x):
    return "x = {0}".format(x)
print(func(10))

```

Использование двух декораторов эквивалентно следующему коду:

```
func = deco1(deco2(func))
```

Здесь сначала будет вызвана функция **deco2()**, а затем функция **deco1()**. Результат выполнения будет присвоен идентификатору **func**.

В качестве ещё одного примера использования декораторов рассмотрим выполнение функции только при правильно введенном пароле:

```

passw = input("Введите пароль: ")
def test_passw(p):
    def deco(f):
        if p == "10": # Сравниваем пароли
            return f
        else:
            return lambda: "Доступ закрыт"
    return deco # Возвращаем функцию-декоратор
@test_passw(passw)
def func():
    return "Доступ открыт"
print(func()) # Вызываем функцию

```

В этом примере после символа **@** указана не ссылка на функцию, а выражение, которое возвращает декоратор. Иными словами, декоратором является не функция **test_passw()**, а результат ее выполнения (функция **deco()**). Если введенный пароль является правильным, то выполнится функция **func()**, в противном случае будет выведена надпись "Доступ закрыт", которую возвращает анонимная функция.

Рекурсия. Вычисление факториала

Рекурсия – это возможность функции вызывать саму себя. Рекурсию удобно использовать для перебора объекта, имеющего заранее неизвестную структуру, или для выполнения неопределенного количества операций. В качестве примера рассмотрим вычисление факториала.

```
def factorial(n):
    if n == 0 or n == 1: return 1
    else:
        return n * factorial(n - 1)
while True:
    x = input("Введите число: ")
    if x.isdigit(): # Если строка содержит только цифры
        x = int(x) # Преобразуем строку в число.
        break # Выходим из цикла
    else:
        print("Вы ввели не число!")
print("Факториал числа {0} = {1}".format(x, factorial(x)))
```

Впрочем, проще всего для вычисления факториала воспользоваться функцией `factorial()` из модуля `math`.

Пример:

```
import math
print( math.factorial(5), math.factorial(6))
(120, 720)
```

Вложенные функции

Одну функцию можно вложить в другую функцию, причем уровень вложенности не ограничен. При этом вложенная функция получает свою собственную локальную область видимости и имеет доступ к переменным, объявленным внутри функции, в которую она вложена (функции-родителя). Рассмотрим вложение функций на примере.

```
def func1(x):
    def func2():
        print(x)
    return func2
f1 = func1(10)
f2 = func1(99)
f1() # Выведет: 10
f2() # Выведет: 99
```

Здесь мы определили функцию **func1()**, принимающую один параметр. Внутри функции **func1()** определена вложенная функция **func2()**. Результатом выполнения функции **func1()** будет ссылка на эту вложенную функцию. Внутри функции **func2()** мы производим вывод значения переменной **x**, которая является локальной в функции **func1()**. Таким образом, помимо локальной, глобальной и

встроенной областей видимости, добавляется *вложенная область видимости*. При этом поиск идентификаторов вначале производится внутри вложенной функции, затем внутри функции-родителя, далее в функциях более высокого уровня и лишь потом в глобальной и встроенных областях видимости. В нашем примере переменная `x` будет найдена в области видимости функции **func1()**.

Следует учитывать, что сохраняются лишь ссылки на переменные, а не их значения в момент определения функции. Например, если после определения функции **func2()** произвести изменение переменной `x`, то будет использоваться это новое значение:

```
def func1(x):
    def func2():
        print(x)
    x = 30
    return func2
f1 = func1(10)
f2 = func1(99)
f1() # Выведет: 30
f2() # Выведет: 30
```

Обратите внимание на результат выполнения. В обоих случаях мы получили значение 30. Если необходимо сохранить именно значение переменной при определении вложенной функции, следует передать значение как значение по умолчанию:

```
def func1(x):
    def func2(x=x): ## Сохраняем текущее значение, а не ссылку
        print(x)
    x = 30
    return func2
f1 = func1(10)
f2 = func1(99)
f1() # Выведет: 10
f2() # Выведет: 99
```

Теперь попробуем из вложенной функции **func2()** изменить значение переменной `x`, объявленной внутри функции **func1()**. Если внутри функции **func2()** присвоить значение переменной `x`, то будет создана новая локальная переменная с таким же именем. Если внутри функции **func2()** объявить переменную как глобальную и присвоить ей значение, то изменится значение глобальной переменной, а не значение переменной `x` внутри функции **func1()**. Таким образом, ни один из изученных ранее способов не позволяет из вложенной функции изменить значение переменной, объявленной внутри функции-родителя. Чтобы решить эту проблему, следует объявить необходимые переменные с помощью ключевого слова **nonlocal**.

```
def func1(a):
    x = a
    def func2(b):
        nonlocal x # Объявляем переменную как nonlocal
```

```

print(x)
x = b # Можем изменить значение x в func1()
return func2
f = func1(10)
f(5) # Выведет: 10
f(12) # Выведет: 5
f(3) # Выведет: 12

```

При использовании ключевого слова **nonlocal** следует помнить, что переменная обязательно должна существовать внутри функции-родителя. В противном случае будет выведено сообщение об ошибке.

Аннотации функций

В **Python 3** функции могут содержать аннотации, которые вводят новый способ документирования. Теперь в заголовке функции допускается указывать предназначение каждого параметра, данные какого типа он может принимать, а также тип возвращаемого функцией значения. Аннотации имеют следующий формат:

```

def <Имя функции>(
    [<Параметр1> [: <Выражение>] [= <Значение по умолчанию>], ...,
    <ПараметрN> [: <Выражение>] [= <Значение по умолчанию>] ] )
    -> <Возвращаемое значение>:
    <Тело функции>

```

В параметрах **<Выражение>** и **<Возвращаемое значение>** можно указать любое допустимое выражение языка **Python**. Это выражение будет выполнено при создании функции. Пример указания аннотаций:

```

def func(a: "Параметр1", b: 10 + 5 = 3) -> None:
    print(a, b)

```

В этом примере для переменной **a** создано описание **"Параметр1"**. Для переменной **b** выражение **10 + 5** является описанием, а число **3** - значением параметра по умолчанию. Кроме того, после закрывающей круглой скобки указан тип возвращаемого функцией значения: **None**. После создания функции все выражения будут выполнены, и результаты сохранятся в виде словаря в атрибуте **__annotations__** объекта функции. Для примера выведем значение этого атрибута:

```

def func(a: "Пример1", b: 10 + 5 = 3) -> None: pass
print(func.__annotations__)
{'a': 'Пример1', 'b': 15, 'return': None}

```

Пример решения задачи на языке программирования Python

Задача. Напишите рекурсивную функцию, которая определяет, сколько раз заданное число встречается в списке.

Решение задачи:

```

def count_occurrences(lst, x, i=0):

```

```
# базовый случай: дошли до конца списка
if i == len(lst):
    return 0
# если текущий элемент равен x
if lst[i] == x:
    return 1 + count_occurrences(lst, x, i + 1)
else:
    return count_occurrences(lst, x, i + 1)
# пример использования
a = [1, 2, 3, 2, 4, 2, 5]
print(count_occurrences(a, 2)) # 3
```